

Assignment 1 - Matrix-matrix multiplication

```
#Importing modules
import numpy as np
import matplotlib.pyplot as plt
import scipy
from timeit import timeit
```

Part 1: a better function

```
def slow_matrix_product(mat1, mat2):
    """Multiply two matrices."""
    assert mat1.shape[1] == mat2.shape[0]
    result = []
    for c in range(mat2.shape[1]):
        column = []
        for r in range(mat1.shape[0]):
            value = 0
            for i in range(mat1.shape[1]):
                value += mat1[r, i] * mat2[i, c]
            column.append(value)
        result.append(column)
    return np.array(result).transpose()
```

```
matrix1 = np.random.rand(10, 10)
matrix2 = np.random.rand(10, 10)
```

```
print(slow_matrix_product(matrix1, matrix2))
print(matrix1 @ matrix2)
```

```
[[2.42905536 1.86887891 2.39497314 3.17657404 3.10238483 2.97626792
 2.03808504 2.00672274 1.82872795 2.36070853]
 [2.45645339 2.59408848 2.2629033 3.37834615 2.93701544 3.62888323
 2.72322347 2.22293261 2.56421468 2.15587483]
 [2.70437437 2.13298696 2.87527069 3.46402462 3.33564972 3.53355922
 2.42975137 2.81994256 2.56291464 2.83310004]
 [2.60516076 2.77305805 2.01102919 3.63243548 3.13045428 3.15288241
 2.91290627 2.4898367 2.39915317 2.50660746]
 [2.5685457 2.73381885 2.39537094 3.32558399 3.07555738 2.9796239
 2.6140457 2.42969528 2.79072682 2.37258839]
 [2.20417813 3.1592358 2.03443036 3.19448246 2.62571303 2.90517236
 2.50011138 1.86816364 2.2843304 2.28604017]
 [2.74905297 1.53870772 2.63059102 2.97813886 3.16744536 2.81357564
 1.74253793 2.02102974 1.25940967 2.27670009]
 [1.86039181 2.03958142 1.95865232 2.99531682 2.43772916 3.05036593
 2.34692477 2.10428019 1.98960312 1.89810898]
 [2.0707442 2.52784203 1.98087837 3.00966273 2.80644909 2.78504884
 2.12931317 1.64517013 2.36047612 2.06280608]
 [2.57682695 2.42291144 2.94410306 3.53034295 3.24282059 3.58120564
 2.42904274 2.67438754 2.05104683 2.30606885]]
[[2.42905536 1.86887891 2.39497314 3.17657404 3.10238483 2.97626792
 2.03808504 2.00672274 1.82872795 2.36070853]
 [2.45645339 2.59408848 2.2629033 3.37834615 2.93701544 3.62888323
 2.72322347 2.22293261 2.56421468 2.15587483]
 [2.70437437 2.13298696 2.87527069 3.46402462 3.33564972 3.53355922
 2.42975137 2.81994256 2.56291464 2.83310004]
 [2.60516076 2.77305805 2.01102919 3.63243548 3.13045428 3.15288241
 2.91290627 2.4898367 2.39915317 2.50660746]
 [2.5685457 2.73381885 2.39537094 3.32558399 3.07555738 2.9796239
 2.6140457 2.42969528 2.79072682 2.37258839]
 [2.20417813 3.1592358 2.03443036 3.19448246 2.62571303 2.90517236
 2.50011138 1.86816364 2.2843304 2.28604017]
 [2.74905297 1.53870772 2.63059102 2.97813886 3.16744536 2.81357564
 1.74253793 2.02102974 1.25940967 2.27670009]
 [1.86039181 2.03958142 1.95865232 2.99531682 2.43772916 3.05036593
 2.34692477 2.10428019 1.98960312 1.89810898]
 [2.0707442 2.52784203 1.98087837 3.00966273 2.80644909 2.78504884
 2.12931317 1.64517013 2.36047612 2.06280608]
 [2.57682695 2.42291144 2.94410306 3.53034295 3.24282059 3.58120564
 2.42904274 2.67438754 2.05104683 2.30606885]]
```

```
def faster_matrix_product(mat1, mat2):
    """
    Compute the matrix product of mat1 and mat2 more efficiently.

    Parameters
    mat1 : ndarray
        Left-hand side matrix of shape (n, m).
    mat2 : ndarray
        Right-hand side matrix of shape (m, p).
```

```

Returns
mat3 : ndarray
    The resulting matrix product of shape (n, p).
"""
# Validate inner dimensions for matrix multiplication
assert mat1.shape[1] == mat2.shape[0], "Incompatible matrix dimensions."

# Preallocate result matrix for efficiency
result = np.zeros((mat1.shape[0], mat2.shape[1]))

# Compute each entry using vectorized dot products between rows and columns
for i in range(mat1.shape[0]):
    for j in range(mat2.shape[1]):
        result[i, j] = np.dot(mat1[i, :], mat2[:, j])

return result

```

```

#Check that it is computing the correct results
#Testing slow_matrix_product
matrix1 = np.random.rand(3, 3)
matrix2 = np.random.rand(3, 3)

print("Slow matrix product:")
print(slow_matrix_product(matrix1, matrix2))

print("\nNumPy @ result:")
print(matrix1 @ matrix2)

```

```

Slow matrix product:
[[1.2000738  1.05327443 0.37895291]
 [0.91149997 0.74336305 0.57284462]
 [0.98388833 0.41306284 0.29878238]]

NumPy @ result:
[[1.2000738  1.05327443 0.37895291]
 [0.91149997 0.74336305 0.57284462]
 [0.98388833 0.41306284 0.29878238]]

```

```

#Checking faster_matrix_product

#Generate random matrices for different sizes
mat2_a = np.random.rand(2, 2)
mat2_b = np.random.rand(2, 2)

mat3_a = np.random.rand(3, 3)
mat3_b = np.random.rand(3, 3)

mat4_a = np.random.rand(4, 4)
mat4_b = np.random.rand(4, 4)

mat5_a = np.random.rand(5, 5)
mat5_b = np.random.rand(5, 5)

```

```

# Compare results using assert statements

print("Checking 2x2 matrices...")
assert np.allclose(mat2_a @ mat2_b, faster_matrix_product(mat2_a, mat2_b))
print("2x2 check passed")

print("\nChecking 3x3 matrices...")
assert np.allclose(mat3_a @ mat3_b, faster_matrix_product(mat3_a, mat3_b))
print("3x3 check passed")

print("\nChecking 4x4 matrices...")
assert np.allclose(mat4_a @ mat4_b, faster_matrix_product(mat4_a, mat4_b))
print("4x4 check passed")

print("\nChecking 5x5 matrices...")
assert np.allclose(mat5_a @ mat5_b, faster_matrix_product(mat5_a, mat5_b))
print("5x5 check passed")

print("\nAll checks passed successfully.")

```

```

Checking 2x2 matrices...
2x2 check passed

Checking 3x3 matrices...
3x3 check passed

Checking 4x4 matrices...

```

4x4 check passed

Checking 5x5 matrices...
5x5 check passed

All checks passed successfully.

Why is faster_matrix_product better than slow_matrix_product?

The faster function uses NumPy's optimized dot product for row and column operations, which runs much quicker than looping through each element in Python.

The slow version uses three nested loops so its runtime grows roughly with N^3 , while the faster version reduces overhead by using vectorized operations and avoids the extra transpose at the end.

```
# Define a range of matrix sizes (about 15 values for an informative plot)
matrix_sizes = np.linspace(50, 950, 15, dtype=int)
print("Matrix sizes tested:", matrix_sizes)
```

```
Matrix sizes tested: [ 50 114 178 242 307 371 435 500 564 628 692 757 821 885 950]
```

```
# Measure execution times for each function
```

```
slow_results = []
fast_results = []
```

```
for n in matrix_sizes:
    A = np.random.rand(n, n)
    B = np.random.rand(n, n)

    # Record the time for one run of each function
    time_slow = timeit('slow_matrix_product(A, B)', globals=globals(), number=1)
    time_fast = timeit('faster_matrix_product(A, B)', globals=globals(), number=1)

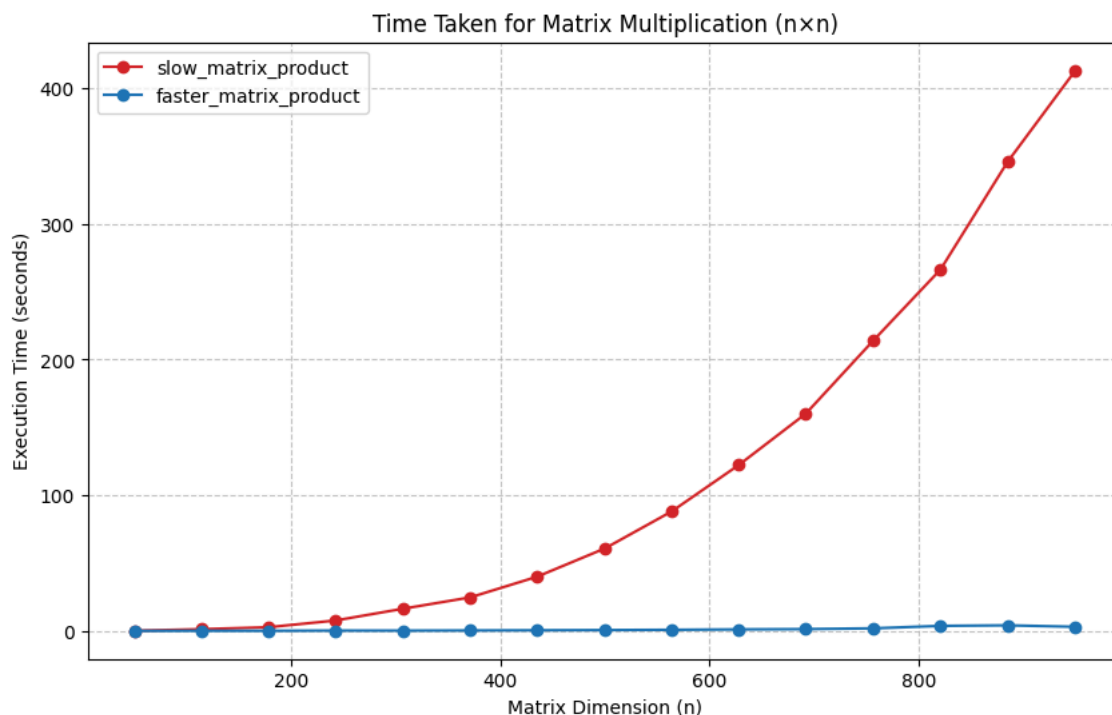
    slow_results.append(time_slow)
    fast_results.append(time_fast)
```

```
# Plot the timing comparison
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(matrix_sizes, slow_results, 'o-', label='slow_matrix_product', color='tab:red')
plt.plot(matrix_sizes, fast_results, 'o-', label='faster_matrix_product', color='tab:blue')
```

```
plt.title("Time Taken for Matrix Multiplication (n×n)")
plt.xlabel("Matrix Dimension (n)")
plt.ylabel("Execution Time (seconds)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Part 2: speeding it up with Numba

```
# Importing numba
from numba import jit, njit

@jit(fastmath=True) # JIT version; allows Python fallback if needed
def faster_matrix_product_jit(mat1, mat2):
    """
    Multiplies two matrices mat1 @ mat2 using @jit.
    """
    assert mat1.shape[1] == mat2.shape[0] # Ensure multiplication is possible

    # Correct output shape: (rows of A, cols of B)
    result = np.zeros((mat1.shape[0], mat2.shape[1]))

    # Loop over rows of A and columns of B
    for i in range(mat1.shape[0]):
        for j in range(mat2.shape[1]):
            result[i, j] = np.dot(mat1[i, :], mat2[:, j])
    return result

@njit(fastmath=True) # NJIT (nopython) version; fully compiled
def faster_matrix_product_njit(mat1, mat2):
    """
    Multiplies two matrices mat1 @ mat2 using @njit.
    """
    assert mat1.shape[1] == mat2.shape[0]

    result = np.zeros((mat1.shape[0], mat2.shape[1]))
    for i in range(mat1.shape[0]):
        for j in range(mat2.shape[1]):
            result[i, j] = np.dot(mat1[i, :], mat2[:, j])
    return result
```

```
# Calling JIT and NJIT functions to skip slow first-time compiling
matrix_a = np.random.rand(32, 32)
matrix_b = np.random.rand(32, 32)
_ = faster_matrix_product_jit(matrix_a, matrix_b)
_ = faster_matrix_product_njit(matrix_a, matrix_b)
```

[Show hidden output](#)

```
# choose 15 sizes
sizes = np.linspace(100, 950, 15, dtype=int)

numpy_times = []
fast_times = []
fast_jit_times = []
fast_njit_times = []

for i in sizes:
    matrix_a = np.random.rand(i, i)
    matrix_b = np.random.rand(i, i)

    next_val_numpy = timeit('matrix_a @ matrix_b', globals=globals(), number=1)
    next_val_fast = timeit('faster_matrix_product(matrix_a, matrix_b)', globals=globals(), number=1)
    next_val_fast_j = timeit('faster_matrix_product_jit(matrix_a, matrix_b)', globals=globals(), number=1)
    next_val_fast_nj = timeit('faster_matrix_product_njit(matrix_a, matrix_b)', globals=globals(), number=1) # <-- fixed

    numpy_times.append(next_val_numpy)
    fast_times.append(next_val_fast)
    fast_jit_times.append(next_val_fast_j)
    fast_njit_times.append(next_val_fast_nj)
```

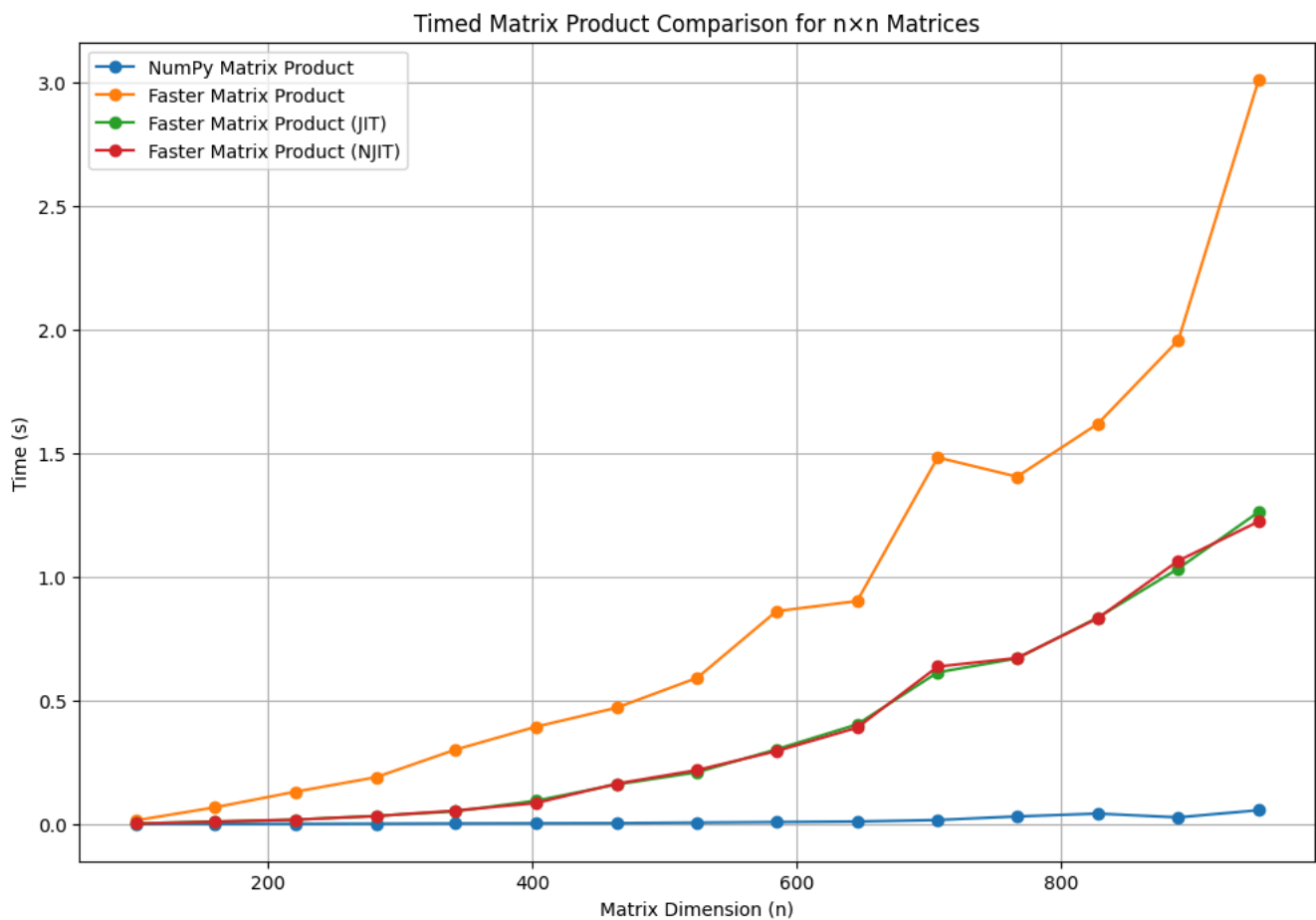
```
fig2 = plt.figure(figsize=(12, 8))
ax1 = fig2.add_subplot(111)

ax1.plot(sizes, numpy_times, '-o', label="NumPy Matrix Product")
ax1.plot(sizes, fast_times, '-o', label="Faster Matrix Product")
ax1.plot(sizes, fast_jit_times, '-o', label="Faster Matrix Product (JIT)")
ax1.plot(sizes, fast_njit_times, '-o', label="Faster Matrix Product (NJIT)")

ax1.set_title("Timed Matrix Product Comparison for n×n Matrices")
ax1.set_xlabel("Matrix Dimension (n)")
ax1.set_ylabel("Time (s)")

plt.legend()
```

```
plt.grid(True)
plt.show()
```



Compare different memory layouts (C and Fortran) using the JIT-compiled function, since JIT and NJIT give nearly identical performance.
import time

```
def time_call(fn, A, B, repeat=3):
    best = float("inf")
    for _ in range(repeat):
        t0 = time.perf_counter()
        C = fn(A, B)
        t1 = time.perf_counter()
        _ = float(np.sum(C)) # touch result
        best = min(best, t1 - t0)
    return best

cc_times, cf_times, fc_times, ff_times = [], [], [], []

for n in sizes:
    # fresh random bases
    A_base = np.random.rand(n, n)
    B_base = np.random.rand(n, n)

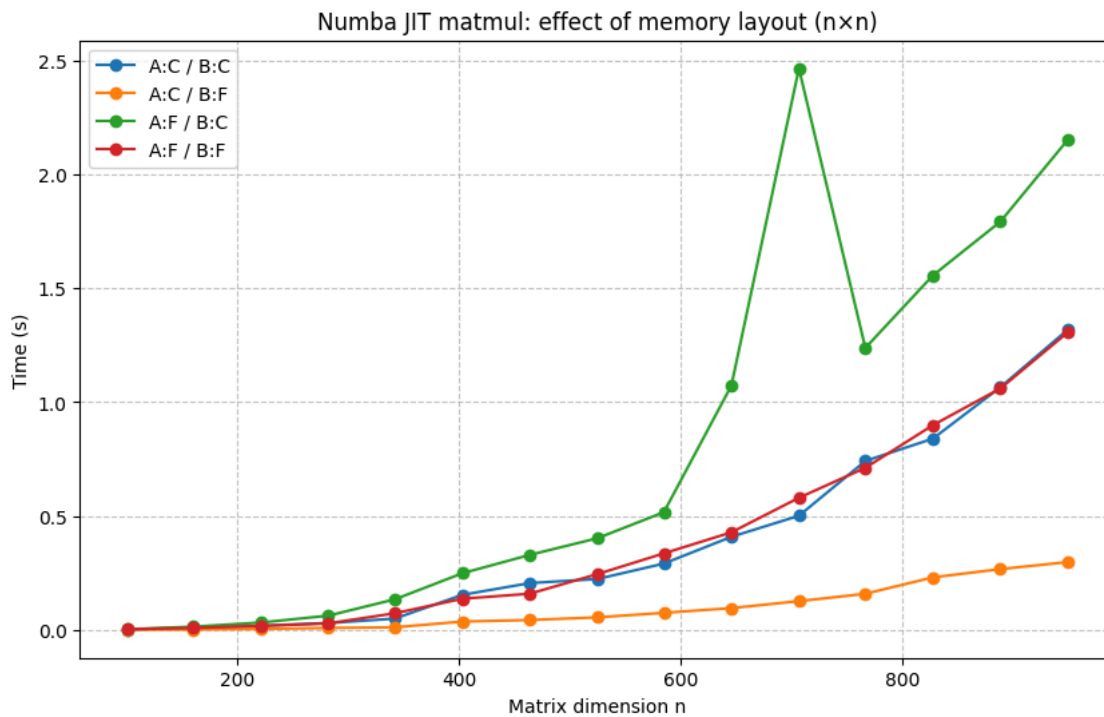
    # make C- and F-ordered copies explicitly
    A_c = np.ascontiguousarray(A_base)
    A_f = np.asfortranarray(A_base)
    B_c = np.ascontiguousarray(B_base)
    B_f = np.asfortranarray(B_base)

    cc_times.append(time_call(faster_matrix_product_njit, A_c, B_c))
    cf_times.append(time_call(faster_matrix_product_njit, A_c, B_f))
    fc_times.append(time_call(faster_matrix_product_njit, A_f, B_c))
    ff_times.append(time_call(faster_matrix_product_njit, A_f, B_f))
```

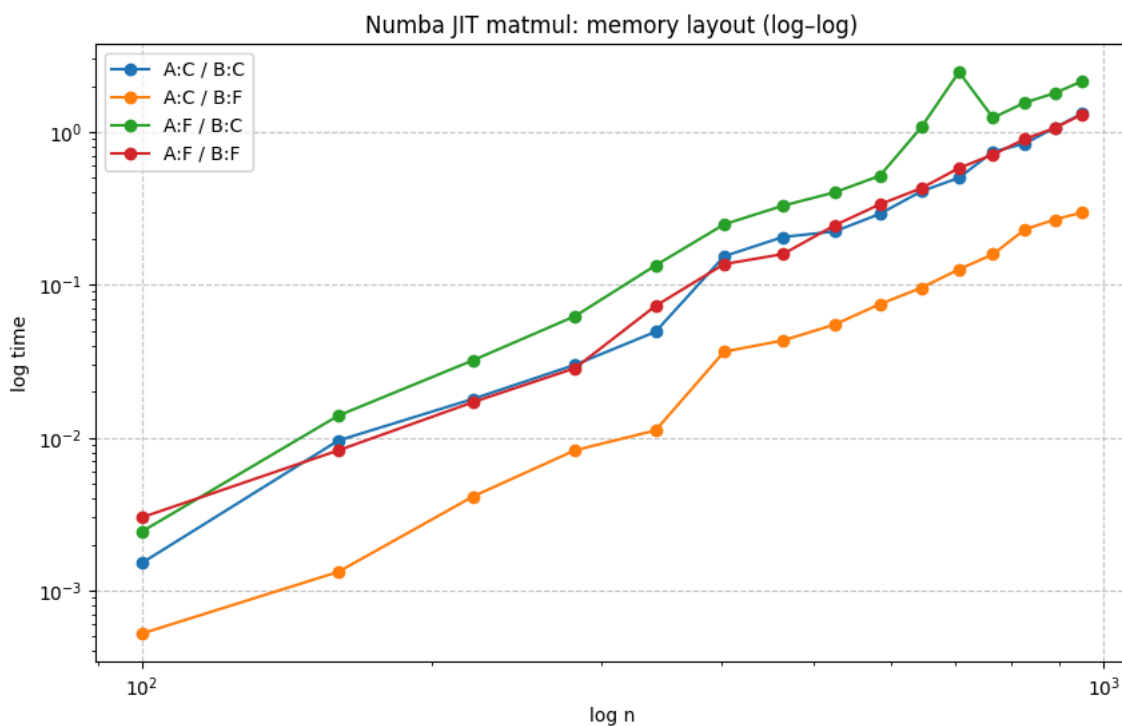
[Show hidden output](#)

```
# Linear scale plot
plt.figure(figsize=(10, 6))
plt.plot(sizes, cc_times, 'o-', label='A:C / B:C')
plt.plot(sizes, cf_times, 'o-', label='A:C / B:F')
plt.plot(sizes, fc_times, 'o-', label='A:F / B:C')
plt.plot(sizes, ff_times, 'o-', label='A:F / B:F')
plt.title("Numba JIT matmul: effect of memory layout (n×n)")
plt.xlabel("Matrix dimension n")
```

```
plt.ylabel("Time (s)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



```
# Log-log plot (helpful when curves grow ~n^3)
plt.figure(figsize=(10, 6))
plt.plot(sizes, cc_times, 'o-', label='A:C / B:C')
plt.plot(sizes, cf_times, 'o-', label='A:C / B:F')
plt.plot(sizes, fc_times, 'o-', label='A:F / B:C')
plt.plot(sizes, ff_times, 'o-', label='A:F / B:F')
plt.xscale('log'); plt.yscale('log')
plt.title("Numba JIT matmul: memory layout (log-log)")
plt.xlabel("log n")
plt.ylabel("log time")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



The fastest runtime occurs when Matrix A uses C-style ordering and Matrix B uses Fortran-style ordering. This is because A's rows and B's columns are stored in contiguous memory blocks, allowing the processor to fetch data sequentially during each multiplication. For

instance, with $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, their memory layouts (A: 1, 2, 3, 4; B: 5, 7, 6, 8) line up efficiently when computing the first row of A with the first column of B. The log plot shows the same overall slope, meaning they scale similarly, but the timing differences come from how the data is stored.